

RAJALAKSHMI ENGINEERING COLLEGE

VISION & MISSION

OF

RAJALAKSHMI ENGINEERING COLLEGE

Vision

To be an institution of excellence in Engineering, Technology and Management Education & Research. To provide competent and ethical professionals with a concern for society.

Mission

To impart quality technical education imbibed with proficiency and humane values. To provide right ambience and opportunities for the students to develop into creative, talented and globally competent professionals. To promote research and development in technology and management for the benefit of the society.

RAJALAKSHMI ENGINEERING COLLEGE

B.Tech. ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Vision

To become a global leader in Artificial Intelligence and Data Science by achieving through excellence in teaching, training, and research, to serve the society.

Mission

- To develop students' skills in innovation, problem-solving, and professionalism through the guidance of well-trained faculty.
- To encourage research activities among students and faculty members to address the evolving challenges of industry and society.
- To impart qualities such as moral and ethical values, along with a commitment to lifelong learning

PROGRAMME EDUCATIONAL OBJECTIVES (PEO)

PEO 1: Build a successful professional career across industry, government, and academia by leveraging technology to develop innovative solutions for real-world problems.

PEO 2: Maintain a learning mindset to continuously enhance knowledge through experience, formal education, and informal learning opportunities.

PEO 3: Demonstrate an ethical attitude while excelling in communication, management, teamwork, and leadership skills

PEO 4: Utilize engineering, problem-solving, and critical thinking skills to drive social, economic, and sustainable impact.

PROGRAMME OUTCOMES (POs)

PO1: Engineering knowledge: Apply the knowledge of Mathematics, Science, Engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12:Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

A graduate of the Artificial Intelligence and Data Science Learning Program will demonstrate

PSO 1: Foundation Skills: Ability to understand, analyze and develop computer programs in the areas related to algorithms, system software, web design, AI, machine learning, deep learning, data science, and networking for efficient design of computer-based systems of varying complexity. Familiarity and practical competence with a broad range of programming language, tools and open source platforms.

PSO 2: Problem-Solving Skills: Ability to apply mathematical methodologies to solve computational task, model real world problem using appropriate AI and ML algorithms. To understand the standard practices and strategies in project development, using open-ended programming environments to deliver a quality product.

PSO 3: Successful Progression: Ability to apply knowledge in various domains to identify research gaps and to provide solution to new ideas, inculcate passion towards higher studies, creating innovative career paths to be an entrepreneur and evolve as an ethically social responsible AI and ML professional.

Subject Code	Subject Name (Lab oriented Theory Courses)	Category	L	T	P	C
AI23531	DEEP LEARNING	PC	3	0	2	4

Objectives:

•	To introduce the foundational concepts of neural networks and enable students to implement a basic three-layer neural network for handwritten digit recognition.
•	To impart knowledge on various training techniques including optimization algorithms and hyperparameter tuning.
•	To familiarize students with convolutional neural networks (CNNs) and guide them through building and experimenting with CNN architectures.
•	To explore recurrent neural networks (RNNs) and their applications in natural language processing, image generation, and autoencoders.
•	To understand generative models, including variational autoencoders (VAE) and generative adversarial networks (GANs), and to discuss best practices for model optimization and scaling.

UNIT-I	**INTRODUCTION TO DEEP LEARNING**	**9**
Neural Networks – Biological Motivation- Perceptron – Multi-layer Perceptron – Feed Forward Network –BackPropagation -Activation and Loss Functions – Implementing three layer Neural Network - Handwritten Digit Recognition		
UNIT-II	**TRAINING TECHNIQUES**	**9**
Numerical Differentiation – Gradient – Implementing a Training Algorithm - Stochastic Gradient Descent – Momentum – AdaGrad – Adam – Initial Weight Values – Regularization – Hyperparameter optimization – Validating Hyper parameters- - Model ensembling - Scaling up model training.		
UNIT-III	**CONVOLUTIONAL NEURAL NETWORKS**	**9**
Overall Architecture – The convolution layer – The pooling layer – Implementing a CNN- LeNet - ImageNet – AlexNet - VGG – GoogLeNet – ResNet		
UNIT-IV	**RECURRENT NEURAL NETWORKS**	**9**
Recursive Neural Networks – Bidirectional RNNs – Deep Recurrent Networks – Applications: Image Generation, Image Compression, Natural Language Processing. Auto encoder - Complete Auto encoder, Regularized Autoencoder-LSTM		
UNIT-V	**GENERATIVE DEEP LEARNING**	**9**

Generative deep learning: Text generation – Deep dream – Neural style transfer – Generating images with variational autoencoders – Introduction to Generative Adversarial Networks.			
	Total Contact Hours	**:**	**45**

List of Experiments	
1	Design and implement a three-layer neural network from scratch using Python. Train the network using the backpropagation algorithm with appropriate activation and loss functions. Apply the model to recognize handwritten digits. Suggested Dataset: MNIST Dataset
2	Develop a multi-layer perceptron (MLP) for a simple classification task. Experiment with different numbers of hidden layers and activation functions, and evaluate the model's performance using accuracy and loss. Suggested Dataset: Iris Dataset
3	Implement a training algorithm using stochastic gradient descent (SGD) with momentum and compare it with the Adam optimizer. Train both models on a dataset and compare their convergence rates and performance. Suggested Dataset: CIFAR-10 Dataset
4	Implement a Convolutional Neural Network (CNN) from scratch to classify images. Train the network using a dataset of labeled images and evaluate its performance. Visualize the learned filters in the convolution layers. Suggested Dataset: CIFAR-10 Dataset
5	Implement and compare the performance of three popular CNN architectures: VGG, ResNet, and GoogLeNet for classification using Dogs vs. Cats dataset. Suggested Dataset: Dogs vs. Cats dataset.
6	Implement a bidirectional recurrent neural network (RNN) to predict sequences in time-series data. Train the RNN and compare its performance with a traditional feed-forward neural network for sequence-based tasks. Suggested Dataset: Airline Passenger Dataset
7	Build a deep recurrent neural network (RNN) to generate image captions. Combine CNN for image feature extraction with RNN for sequence generation. Suggested Dataset: MS COCO Dataset
8	Implement a variational autoencoder (VAE) to generate new images from a given dataset. Train the VAE to learn the latent representation of images and generate new samples from the learned distribution. Suggested Dataset: CelebA Dataset
9	Build a text generation model using Long Short-Term Memory (LSTM) networks. Train the model on a text corpus to generate coherent sequences of text and evaluate the generated text for fluency and coherence. Suggested Dataset: Shakespeare Corpus

10	Train a Generative Adversarial Network (GAN) to generate new images from a dataset. Evaluate the quality of the images generated using visual inspection and a quantitative metric like the Inception Score (IS) or Fréchet Inception Distance (FID). Suggested Dataset: LSUN Dataset
11	Mini Project • **Choose a Topic**: Identify a deep learning problem of interest, such as image classification, text generation, or anomaly detection. Research related work using platforms like Google Scholar. • **Dataset Selection**: Find or collect a suitable dataset from sources like Kaggle or UCI. Ensure it is relevant, well-sized, and consider preprocessing requirements. • **Develop Methodology**: Start with baseline models, then experiment with advanced architectures (e.g., CNNs, Transformers). Use frameworks like TensorFlow or PyTorch. • **Implementation & Evaluation**: Train models and evaluate performance using appropriate metrics (e.g., accuracy, F1-score). Document findings systematically. **Discuss & Present**: Analyze results, highlight challenges, and present your work with clear insights and future directions.
	Contact Hours : 30
	Total Contact Hours : 75

Course Outcomes:	
On completion of the course, the students will be able to	
•	Define the architecture of a neural network and implement a three-layer neural network
•	Analyze and apply various training techniques to optimize neural network performance
•	Design and evaluate convolutional neural networks (CNNs) by building models for image classification tasks
•	Illustrate the working of recurrent neural networks (RNNs) and apply autoencoders for various applications
•	Construct deep generative model for various applications.
Text Books:	
1	Koki Saitoh, "Deep Learning from the Basics - Python and Deep Learning: Theory and Implementation", 1st edition, Packt Publishing,2021.
2	Ian Goodfellow, Yoshua Bengio and Aaron Courville," Deep Learning ",1st edition ,MIT Press,2016.

AI23531 **DEEP LEARNING**

LAB PLAN

EXERCISE NO	EXERCISE NAME	REQUIRED HOURS
1	Three-Layer Neural Network From Scratch	3
2	Multi-Layer Perceptron (Mlp) For Classification	3
3	Comparison Of Sgd With Momentum Vs Adam Optimizer	3
4	Implementation Of A Convolutional Neural Network (Cnn)	3
5	Comparative Analysis Of Vgg, Resnet, And Googlenet	3
6	Bidirectional Rnn Vs Feedforward Nn For Time-Series Prediction	3
7	Image Captioning Using Cnn-Rnn Architecture	3
8	Image Generation Using Variational Autoencoder (Vae)	3
9	Text Generation Using Lstm Networks	3
10	Image Generation Using Generative Adversarial Network (Gan)	3

CO - PO – PSO matrices of course

PO/PSO CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
AI23531.1	**3**	**3**	**2**	**2**	**1**	**-**	**-**	**1**	**-**	**-**	**-**	**1**	**3**	**3**	**1**
AI23531.2	**3**	**3**	**2**	**2**	**2**	**-**	**-**	**2**	**-**	**-**	**-**	**1**	**3**	**3**	**2**
AI23531.3	**3**	**3**	**3**	**2**	**2**	**-**	**-**	**2**	**-**	**-**	**-**	**1**	**3**	**3**	**1**
AI23531.4	**3**	**3**	**3**	**2**	**3**	**1**	**1**	**2**	**1**	**1**	**-**	**1**	**3**	**3**	**3**
AI23531.5	**3**	**3**	**3**	**2**	**3**	**1**	**1**	**2**	**1**	**1**	**-**	**1**	**3**	**3**	**3**
Average	**3**	**3**	**2.6**	**2**	**2.2**	**1**	**1**	**1.8**	**1**	**1**	**-**	**1**	**3**	**3**	**2**

Correlation levels 1, 2 or 3 are as defined below:
1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High)
No correlation: "-"

EX 1 THREE-LAYER NEURAL NETWORK FROM SCRATCH

DATE:

Problem Statement:

Design and implement a three-layer neural network from scratch using Python. Train the network using the backpropagation algorithm with appropriate activation and loss functions. Apply the model to recognize handwritten digits using the MNIST dataset.

Objectives:

1. Understand the structure and working of a basic feedforward neural network.
2. Implement forward propagation and backpropagation from scratch.
3. Use the sigmoid activation function and cross-entropy loss.
4. Train the model on the MNIST dataset and evaluate the loss over epochs.
5. Visualize training performance and perform predictions on sample images.

Scope:

This experiment demonstrates the core principles behind neural networks and training them using backpropagation. It is foundational for understanding deep learning and how more complex models (like CNNs or Transformers) build upon this architecture.

Tools and Libraries Used:

1. Python 3.x
2. NumPy
3. Matplotlib
4. TensorFlow (for dataset loading)
5. scikit-learn (OneHotEncoder)

Implementation Steps:

Step 1: Import Necessary Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from sklearn.preprocessing import OneHotEncoder
```

Step 2: Load and Preprocess the Dataset

```
(X_train, y_train), (_, _) = mnist.load_data()
X_train = X_train.reshape(-1, 784) / 255.0
encoder = OneHotEncoder(sparse_output=False)
y_train = encoder.fit_transform(y_train.reshape(-1, 1))
```

Step 3: Initialize the Network

```
input_size, hidden_size, output_size = 784, 64, 10
W1 = np.random.randn(input_size, hidden_size) * 0.01
b1 = np.zeros((1, hidden_size))
W2 = np.random.randn(hidden_size, output_size) * 0.01
b2 = np.zeros((1, output_size))
```

Step 4: Define Activation and Loss Functions

```
sigmoid = lambda x: 1 / (1 + np.exp(-x))
sigmoid_deriv = lambda x: x * (1 - x)
loss_fn = lambda y, y_hat: -np.mean(y * np.log(y_hat + 1e-8))
```

Step 5: Train the Model

```
epochs, lr = 10, 0.1
losses = []
for epoch in range(epochs):
    total_loss = 0
    for i in range(X_train.shape[0]):
        x = X_train[i:i+1]
        y = y_train[i:i+1]

        z1 = x @ W1 + b1
        a1 = sigmoid(z1)
        z2 = a1 @ W2 + b2
        a2 = sigmoid(z2)
        loss = loss_fn(y, a2)
        total_loss += loss

        dz2 = a2 - y
        dW2 = a1.T @ dz2
        db2 = dz2

        dz1 = (dz2 @ W2.T) * sigmoid_deriv(a1)
        dW1 = x.T @ dz1
        db1 = dz1

        W2 -= lr * dW2
        b2 -= lr * db2
        W1 -= lr * dW1
        b1 -= lr * db1
    losses.append(total_loss / X_train.shape[0])
    print(f"Epoch {epoch+1}, Loss: {losses[-1]:.4f}")
```

Step 6: Visualize Training Loss

```
plt.plot(losses)
```

```
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

Step 7: Predict a Sample Digit

```
def predict(img):
    img = img.reshape(1, 784) / 255.0
    a1 = sigmoid(img @ W1 + b1)
    a2 = sigmoid(a1 @ W2 + b2)
    return np.argmax(a2)

idx = 100
plt.imshow(X_train[idx].reshape(28, 28), cmap='gray')
plt.title(f"Prediction: {predict(X_train[idx])}")
plt.axis('off')
plt.show()
```

Conclusion:

This experiment successfully demonstrates the manual implementation of a basic three-layer neural network. Through backpropagation and weight updates using gradient descent, the model learns to classify handwritten digits. It emphasizes key deep learning concepts such as feedforward computation, loss minimization, and numerical stability in gradient calculations.

EX 2 MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION

DATE:

Problem Statement:

Develop a Multi-Layer Perceptron (MLP) for a simple classification task. Experiment with different numbers of hidden layers and activation functions, and evaluate the model's performance using accuracy and loss.

Suggested Dataset: Iris Dataset

Objectives:

1. Understand the structure and purpose of MLPs for classification.
2. Experiment with various hidden layer configurations and activation functions.
3. Train the MLP using the Iris dataset and evaluate its accuracy and loss.
4. Visualize training progress and use the trained model for prediction.

Scope:

This experiment provides insights into how neural networks with multiple layers (MLPs) perform on structured classification tasks. It demonstrates model tuning using different architectures and activation functions, an essential concept in designing effective deep learning models.

Tools and Libraries Used:

1. Python 3.x
2. TensorFlow / Keras
3. scikit-learn (for data preprocessing and dataset loading)
4. Matplotlib (for visualization)

Implementation Steps:

Step 1: Import Necessary Libraries

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Step 2: Load and Preprocess Data

```
iris = load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)
```

```
classes = iris.target_names

encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.2, random_state=42)
```

Step 3: Define MLP Model Creation Function

```
def create_mlp(input_dim, output_dim, hidden_layers, activation='relu'):
    model = Sequential()
    model.add(Dense(hidden_layers[0], input_dim=input_dim, activation=activation))
    for units in hidden_layers[1:]:
        model.add(Dense(units, activation=activation))
    model.add(Dense(output_dim, activation='softmax'))
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

Step 4: Train and Evaluate MLP with Various Configurations

```
hidden_layer_configs = [[8], [16, 8], [32, 16, 8]]
activations = ['relu', 'tanh', 'sigmoid']

for hidden_layers in hidden_layer_configs:
    for activation in activations:
        print(f"\nTesting MLP with hidden_layers={hidden_layers}, activation={activation}")
        model = create_mlp(input_dim=4, output_dim=3, hidden_layers=hidden_layers, activation=activation)
        history = model.fit(X_train, y_train, epochs=50, batch_size=5, verbose=0, validation_split=0.1)
        test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
        print(f"Test Accuracy: {test_acc:.4f}, Test Loss: {test_loss:.4f}")
```

Step 5: Plot Accuracy and Loss Curves

```
        # Plot accuracy and loss
        plt.figure(figsize=(10, 4))
        plt.suptitle(f"Config: {hidden_layers}, Activation: {activation}", fontsize=14)

        plt.subplot(1, 2, 1)
        plt.plot(history.history['accuracy'], label='Train Acc')
        plt.plot(history.history['val_accuracy'], label='Val Acc')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
```

```
plt.title('Model Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Model Loss')
plt.legend()

plt.tight_layout()
plt.show()
```

Step 6: Predict on New Input

```
sepal_length = float(input("Sepal length (cm): "))
sepal_width = float(input("Sepal width (cm): "))
petal_length = float(input("Petal length (cm): "))
petal_width = float(input("Petal width (cm): "))

user_input = np.array([[sepal_length, sepal_width, petal_length, petal_width]])
user_input_scaled = scaler.transform(user_input)

# Predict using the last trained model
prediction = model.predict(user_input_scaled)
predicted_class_index = np.argmax(prediction)
predicted_class_name = classes[predicted_class_index]

print(f"\n🌸 Predicted Iris Species: {predicted_class_name}")
```

Conclusion:

This experiment demonstrates the use of MLPs for a multi-class classification problem using the Iris dataset. Through experimentation with different hidden layer depths and activation functions, students gain a practical understanding of neural network design and optimization techniques.

EX 3 COMPARISON OF SGD WITH MOMENTUM VS ADAM OPTIMIZER

DATE:

Problem Statement:

Implement a training algorithm using Stochastic Gradient Descent (SGD) with momentum and compare it with the Adam optimizer. Train both models on a dataset and compare their convergence rates and performance.

Suggested Dataset: CIFAR-10

Objectives:

1. Understand the principles of optimization algorithms in deep learning.
2. Implement and train models using SGD with momentum and Adam.
3. Analyze and compare the learning behavior and convergence patterns of the two optimizers.
4. Visualize loss and accuracy across epochs for both optimization methods.

Scope:

This experiment gives students a comparative understanding of two widely used optimization strategies: SGD with momentum and Adam. Using a basic MLP and the CIFAR-10 dataset, students will learn the impact of optimizer choice on model convergence and final accuracy.

Tools and Libraries Used:

1. Python 3.x
2. PyTorch
3. Matplotlib
4. torchvision (for CIFAR-10 dataset)

Implementation Steps:

Step 1: Import Necessary Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

Step 2: Set Device and Load Dataset

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([
   transforms.ToTensor(),
   transforms.Normalize((0.5,), (0.5,))
```

```
])
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, transform=transform, download=True)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True)
```

Step 3: Define MLP Model

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.net = nn.Sequential(
            nn.Linear(3*32*32, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.net(x)
```

Step 4: Define Training Function

```
def train(model, optimizer, epochs=10):
    model.to(device)
    loss_fn = nn.CrossEntropyLoss()
    losses = []
    accuracies = []

    for epoch in range(epochs):
        total_loss = 0
        correct = 0
        total = 0
        model.train()

        for imgs, labels in trainloader:
            imgs, labels = imgs.to(device), labels.to(device)

            outputs = model(imgs)
            loss = loss_fn(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
```

```
        avg_loss = total_loss / len(trainloader)
        accuracy = 100.0 * correct / total

        losses.append(avg_loss)
        accuracies.append(accuracy)

        print(f"Epoch {epoch+1}: Loss = {avg_loss:.4f}, Accuracy = {accuracy:.2f}%")

    return losses, accuracies
```

Step 5: Train with SGD + Momentum and with Adam

```
model_sgd = MLP()
sgd = optim.SGD(model_sgd.parameters(), lr=0.01, momentum=0.9)
losses_sgd, acc_sgd = train(model_sgd, sgd)
model_adam = MLP()
adam = optim.Adam(model_adam.parameters(), lr=0.001)
losses_adam, acc_adam = train(model_adam, adam)
```

Step 6: Visualize Loss and Accuracy Comparison

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(losses_sgd, label="SGD + Momentum")
plt.plot(losses_adam, label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Comparison on CIFAR-10 (MLP)")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(acc_sgd, label="SGD + Momentum")
plt.plot(acc_adam, label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison on CIFAR-10 (MLP)")
plt.legend()
plt.tight_layout()
plt.show()
```

Conclusion:

This experiment shows a practical implementation of two optimization techniques and highlights their impact on model performance. Adam typically converges faster due to adaptive learning rates, while SGD with momentum offers more controlled, gradual learning. Choosing the right optimizer depends on the nature of the data and the task complexity.

EX 4 IMPLEMENTATION OF A CONVOLUTIONAL NEURAL NETWORK (CNN)

DATE:

Problem Statement:

Implement a Convolutional Neural Network (CNN) from scratch using PyTorch to classify images. Train the network using a dataset of labeled images and evaluate its performance. Additionally, visualize the learned filters in the convolution layers.

Suggested Dataset: CIFAR-10

Objectives:

1. Understand the architecture and functionality of Convolutional Neural Networks (CNNs).
2. Implement CNN layers including convolution, pooling, and fully connected layers.
3. Train the model on the CIFAR-10 dataset and evaluate its performance.
4. Visualize the learned filters to interpret feature extraction at early layers.

Scope:

CNNs are powerful architectures for image classification tasks. This experiment helps students grasp key CNN concepts such as spatial feature learning, hierarchical representation, and how filters learn patterns in images. Visualizing filters bridges the gap between model architecture and interpretability.

Tools and Libraries Used:

1. Python 3.x
2. PyTorch
3. torchvision
4. Matplotlib

Implementation Steps:

Step 1: Load and Preprocess CIFAR-10 Dataset

```
import torchvision.transforms as transforms
import torchvision

transform = transforms.Compose([
   transforms.ToTensor(),
   transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform)
```

```
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)

testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=64, shuffle=False)

classes = trainset.classes
```

Step 2: Define CNN Architecture

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)   # Output: 16x32x32
        self.pool = nn.MaxPool2d(2, 2)                # Output: 16x16x16
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)  # Output: 32x16x16 → 32x8x8 after
pooling
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 32 * 8 * 8)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

Step 3: Train the CNN

```
import torch
import torch.optim as optim

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = SimpleCNN().to(device)

lossfn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(10):
    running_loss = 0.0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = lossfn(outputs, labels)
```

```
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {running_loss / len(trainloader):.4f}")
```

Step 4: Evaluate Model Performance

```
correct, total = 0, 0
with torch.no_grad():
    for images, labels in testloader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

print(f"Accuracy on test data: {100 * correct / total:.2f}%")
```

Step 5: Visualize Learned Filters

```
import matplotlib.pyplot as plt

def visualize_filters(layer, n_filters=8):
    filters = layer.weight.data.clone().cpu()
    fig, axs = plt.subplots(1, n_filters, figsize=(15, 4))
    for i in range(n_filters):
        f = filters[i]
        f = (f - f.min()) / (f.max() - f.min())  # Normalize for display
        axs[i].imshow(f.permute(1, 2, 0))
        axs[i].axis('off')
        axs[i].set_title(f'Filter {i+1}')
    plt.tight_layout()
    plt.show()

visualize_filters(model.conv1)
```

Conclusion:

This experiment successfully demonstrates how CNNs extract hierarchical spatial features for image classification. By training on CIFAR-10, we observed performance improvements and explored how early layers learn to detect basic patterns using visualized filters. This forms the basis for more advanced deep learning models used in computer vision.

EX 5 COMPARATIVE ANALYSIS OF VGG, RESNET, AND GOOGLENET ON IMAGE CLASSIFICATION

DATE:

Problem Statement:

Implement and compare the performance of three popular CNN architectures: VGG, ResNet, and GoogLeNet for image classification. Use a labeled dataset to train each model and evaluate their convergence and accuracy.

Suggested Dataset: Dogs vs. Cats dataset

Objectives:

1. Understand the architectural differences between VGG, ResNet, and GoogLeNet.
2. Train all three models on the same dataset using transfer learning.
3. Analyze and compare performance using validation accuracy.
4. Apply the trained models to predict classes for custom images.

Scope:

This experiment demonstrates the power of transfer learning using pre-trained CNN models. Students explore how architectural changes affect accuracy and generalization. Comparing models under identical training settings aids in choosing the right model for real-world applications.

Tools and Libraries Used:

1. Python 3.x
2. PyTorch
3. torchvision
4. Matplotlib
5. PIL (Python Imaging Library)

Implementation Steps:

Step 1: Import Necessary Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, random_split, Subset
import matplotlib.pyplot as plt
from PIL import Image
import os
```

Step 2: Configure Device and Define Labels

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
```

Step 3: Preprocess Data and Load CIFAR-10

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

dataset = datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform)
test_dataset = datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform)

dataset = Subset(dataset, range(500))
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
```

Step 4: Define Training and Evaluation Function

```
def train_and_evaluate(model, name, num_epochs=5):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=1e-4)

    train_accs, val_accs = [], []

    for epoch in range(num_epochs):
        model.train()
        correct, total = 0, 0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
```

```
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)
        train_accs.append(100 * correct / total)

        model.eval()
        correct, total = 0, 0
        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                _, predicted = outputs.max(1)
                correct += (predicted == labels).sum().item()
                total += labels.size(0)
        val_accs.append(100 * correct / total)

        print(f"{name} Epoch {epoch+1}/{num_epochs} - Train Acc: {train_accs[-1]:.2f}%, Val
Acc: {val_accs[-1]:.2f}%")

    return model, train_accs, val_accs
```

Step 5: Select Pretrained Models and Replace Final Layers

```
def get_model(name):
    if name == "vgg":
        model = models.vgg16(pretrained=True)
        model.classifier[6] = nn.Linear(4096, 10)
    elif name == "resnet":
        model = models.resnet18(pretrained=True)
        model.fc = nn.Linear(model.fc.in_features, 10)
    elif name == "googlenet":
        model = models.googlenet(pretrained=True, aux_logits=True)
        model.fc = nn.Linear(model.fc.in_features, 10)
    else:
        raise ValueError("Unknown model")
    return model
```

Step 6: Train All Models and Collect Results

```
results = {}
trained_models = {}

for model_name in ["vgg", "resnet", "googlenet"]:
    print(f"\n🔷 Training {model_name.upper()} on CIFAR-10...")
    model = get_model(model_name)
    trained_model, train_acc, val_acc = train_and_evaluate(model, model_name.upper(),
num_epochs=5)
    results[model_name] = (train_acc, val_acc)
    trained_models[model_name] = trained_model
```

Step 7: Plot Accuracy Comparison

```
plt.figure(figsize=(10, 6))
for name, (train_acc, val_acc) in results.items():
   plt.plot(val_acc, label=f'{name.upper()} Val Acc')
plt.title('Validation Accuracy Comparison on CIFAR-10')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.grid(True)
plt.show()
```

Step 8: Predict on Custom Image

```
def predict_image(image_path, models_dict):
   image = Image.open(image_path).convert('RGB')
   image = transform(image).unsqueeze(0).to(device)

   print(f"\n🖼 Prediction results for image: {image_path}")
   for model_name, model in models_dict.items():
      model.eval()
      with torch.no_grad():
         outputs = model(image)
         _, predicted = outputs.max(1)
         pred_class = class_names[predicted.item()]
         print(f"{model_name.upper():<10} => {pred_class}")

custom_image_path = "download.jpeg"
if os.path.exists(custom_image_path):
   predict_image(custom_image_path, trained_models)
else:
   print(f"\n❗ Image not found: {custom_image_path}. Please add an image to test.")
```

Conclusion:

This experiment successfully compares three state-of-the-art CNN architectures—VGG16, ResNet18, and GoogLeNet—on an image classification task. The experiment demonstrates how model depth, skip connections (ResNet), and inception modules (GoogLeNet) influence learning capability and accuracy. Such comparisons guide model selection for future deployment tasks.

EX 6 BIDIRECTIONAL RNN VS FEEDFORWARD NN FOR TIME-SERIES PREDICTION

DATE:

Problem Statement:

Implement a Bidirectional Recurrent Neural Network (BiRNN) to predict sequences in time-series data. Train the model and compare its performance with a traditional Feedforward Neural Network (FFNN) for sequence-based tasks.

Suggested Dataset: Airline Passenger Dataset

Objectives:

1. Understand the application of RNNs and FFNNs for time-series forecasting.
2. Train a bidirectional RNN model to capture sequential dependencies.
3. Compare prediction performance with a feedforward neural network.
4. Visualize and evaluate predictions using metrics such as Mean Squared Error (MSE).

Scope:

Recurrent Neural Networks are well-suited for tasks involving sequential data. This experiment demonstrates the power of BiRNNs in modeling time dependencies and compares them with simpler feedforward architectures, providing insight into the role of model memory in sequence modeling.

Tools and Libraries Used:

1. Python 3.x
2. pandas
3. numpy
4. matplotlib
5. scikit-learn
6. PyTorch

Implementation Steps:

Step 1: Import Necessary Libraries

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

Step 2: Data Preparation

```
url='https://raw.githubusercontent.com/jbrownlee/Datasets/refs/heads/master/monthly-airline-passengers.csv'
```

```
df = pd.read_csv(url, usecols=[1])
data = df.values.astype('float32')

scaler = MinMaxScaler(feature_range=(0, 1))
data_scaled = scaler.fit_transform(data)
```

Step 3: Create Sequences for Time-Series Prediction

```
SEQ_LENGTH = 10
X = np.array([data_scaled[i:i+SEQ_LENGTH] for i in range(len(data_scaled) - SEQ_LENGTH)])
y = np.array([data_scaled[i + SEQ_LENGTH] for i in range(len(data_scaled) - SEQ_LENGTH)])

train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]
```

Step 4: Prepare PyTorch Datasets and Loaders

```
import torch
from torch.utils.data import TensorDataset, DataLoader

X_train_tensor = torch.tensor(X_train)
y_train_tensor = torch.tensor(y_train)
X_test_tensor  = torch.tensor(X_test)
y_test_tensor  = torch.tensor(y_test)

train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor), batch_size=16, shuffle=True)
test_loader  = DataLoader(TensorDataset(X_test_tensor, y_test_tensor), batch_size=1)
```

Step 5: Define BiRNN and Feedforward Models

```
import torch.nn as nn

class BiRNN(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=1):
        super(BiRNN, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True, bidirectional=True)
        self.fc = nn.Linear(hidden_size * 2, 1)

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out[:, -1, :])

class FeedforwardNN(nn.Module):
    def __init__(self, input_size):
        super(FeedforwardNN, self).__init__()
```

```
        self.fc1 = nn.Linear(input_size, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 1)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        return self.fc2(self.relu(self.fc1(x)))
```

Step 6: Define Training and Evaluation Functions

```
def train_model(model, loader, epochs=100):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    model.train()
    for epoch in range(epochs):
        loss_epoch = 0
        for seqs, targets in loader:
            optimizer.zero_grad()
            outputs = model(seqs)
            loss = criterion(outputs, targets)
            loss.backward()
            optimizer.step()
            loss_epoch += loss.item()
        if (epoch + 1) % 20 == 0:
            print(f"Epoch {epoch+1}/{epochs}, Loss: {loss_epoch / len(loader):.5f}")

def evaluate_model(model, X):
    model.eval()
    with torch.no_grad():
        return model(X).numpy()
```

Step 7: Train and Predict with Both Models

```
birnn = BiRNN()
train_model(birnn, train_loader)

ffnn = FeedforwardNN(input_size=SEQ_LENGTH)
train_model(ffnn, train_loader)

pred_birnn = evaluate_model(birnn, X_test_tensor)
pred_ffnn = evaluate_model(ffnn, X_test_tensor)
```

Step 8: Inverse Transform and Plot Results

```
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

pred_birnn_inv = scaler.inverse_transform(pred_birnn)
pred_ffnn_inv = scaler.inverse_transform(pred_ffnn)
y_test_inv = scaler.inverse_transform(y_test)
```

```
plt.figure(figsize=(12, 5))
plt.plot(y_test_inv, label='Actual')
plt.plot(pred_birnn_inv, label='BiRNN Prediction')
plt.plot(pred_ffnn_inv, label='FFNN Prediction')
plt.legend()
plt.title('BiRNN vs Feedforward NN on Airline Passenger Data')
plt.xlabel('Time Step')
plt.ylabel('Passengers')
plt.show()

mse_birnn = mean_squared_error(y_test_inv, pred_birnn_inv)
mse_ffnn = mean_squared_error(y_test_inv, pred_ffnn_inv)

print(f"BiRNN MSE: {mse_birnn:.3f}")
print(f"FFNN MSE: {mse_ffnn:.3f}")
```

Conclusion:

This experiment highlights the advantage of BiRNNs in modeling temporal dependencies in sequential data. While feedforward networks treat time steps independently, BiRNNs consider both past and future context, resulting in improved predictive performance. The MSE metric provides a quantitative basis for comparison.

EX 7 IMAGE CAPTIONING USING CNN-RNN ARCHITECTURE

DATE:

Problem Statement:

Build a deep Recurrent Neural Network (RNN) to generate captions for images. Combine a Convolutional Neural Network (CNN) for feature extraction with an RNN for sequence generation.

Objectives:

1. Understand image captioning using encoder-decoder architecture.
2. Use a pre-trained Vision Transformer (ViT) as the CNN encoder and GPT-2 as the RNN decoder.
3. Generate meaningful captions for visual content using beam search decoding.
4. Explore the power of vision-language models for real-world applications.

Scope:

This experiment demonstrates the fusion of computer vision and natural language processing through encoder-decoder architectures. Students gain insight into how visual features are mapped to textual sequences, a key component in modern AI systems such as image tagging, accessibility tech, and content generation.

Tools and Libraries Used:

1. Python 3.x
2. PyTorch
3. HuggingFace Transformers
4. VisionEncoderDecoderModel (ViT + GPT-2)
5. PIL (Python Imaging Library)

Implementation Steps:

Step 1: Import Required Libraries

```
import torch
from transformers import VisionEncoderDecoderModel, ViTImageProcessor,
AutoTokenizer
from PIL import Image
from transformers.utils import hub

# Set extended download timeout
hub.HUGGINGFACE_HUB_HTTP_TIMEOUT = 60
```

Step 2: Load Pretrained Image Captioning Model

```
model = VisionEncoderDecoderModel.from_pretrained("nlpconnect/vit-gpt2-image-captioning")
processor = ViTImageProcessor.from_pretrained("nlpconnect/vit-gpt2-image-captioning")
tokenizer = AutoTokenizer.from_pretrained("nlpconnect/vit-gpt2-image-captioning")

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
model.eval()
```

Step 3: Define Caption Generation Function

```
def generate_caption(image_path, max_length=30, num_beams=4):
    image = Image.open(image_path).convert("RGB")
    pixel_values = processor(images=image, return_tensors="pt").pixel_values.to(device)

    output_ids = model.generate(pixel_values,
                                max_length=max_length,
                                num_beams=num_beams,
                                early_stopping=True)

    caption = tokenizer.decode(output_ids[0], skip_special_tokens=True)
    return caption
```

Step 4: Load Image and Generate Caption

```
if __name__ == "__main__":
    img_path = "download.jpeg"  # Replace with any image file
    print("Caption:", generate_caption(img_path))
```

Conclusion:

This experiment showcases how deep learning models can describe visual scenes with human-like captions. By combining ViT for image encoding and GPT-2 for text generation, we observe effective cross-modal learning. Pre-trained models significantly reduce development time and enable quick prototyping.

EX 8 IMAGE GENERATION USING VARIATIONAL AUTOENCODER (VAE)

DATE:

Problem Statement:

Implement a Variational Autoencoder (VAE) to generate new images from a given dataset. Train the model to learn the latent representation of images and generate new samples from the learned distribution.

Suggested Dataset: CelebA Dataset

Objectives:

1. Understand the concept of generative models using latent space representations.
2. Implement encoder-decoder architecture with reparameterization.
3. Train a VAE on CelebA and generate new human face images.
4. Visualize generated samples from the latent space.

Scope:

VAEs are probabilistic generative models capable of learning latent representations and generating realistic samples. This experiment explores the VAE pipeline—encoding, sampling via reparameterization, and decoding—to generate new samples that resemble the training distribution.

Tools and Libraries Used:

1. Python 3.x
2. PyTorch
3. torchvision
4. matplotlib
5. CelebA Dataset

Implementation Steps:

Step 1: Import Required Libraries

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt
```

Step 2: Configure Parameters and Load Dataset

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

image_size = 64
batch_size = 128
latent_dim = 100
num_epochs = 5
learning_rate = 1e-3

transform = transforms.Compose([
   transforms.CenterCrop(178),
   transforms.Resize(image_size),
   transforms.ToTensor(),
   transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

dataset = datasets.CelebA(root='data', split='train', download=True, transform=transform)
dataset = Subset(dataset, range(500))  # Limit to 500 samples for speed
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

Step 3: Define Encoder Network

```
class Encoder(nn.Module):
   def __init__(self, latent_dim):
      super(Encoder, self).__init__()
      self.conv = nn.Sequential(
         nn.Conv2d(3, 64, 4, 2, 1), nn.ReLU(),
         nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),
         nn.Conv2d(128, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.ReLU(),
         nn.Conv2d(256, 512, 4, 2, 1), nn.BatchNorm2d(512), nn.ReLU()
      )
      self.fc_mu = nn.Linear(512*4*4, latent_dim)
      self.fc_logvar = nn.Linear(512*4*4, latent_dim)

   def forward(self, x):
      x = self.conv(x)
      x = x.view(x.size(0), -1)
      return self.fc_mu(x), self.fc_logvar(x)
```

Step 4: Define Decoder Network

```
class Decoder(nn.Module):
   def __init__(self, latent_dim):
      super(Decoder, self).__init__()
      self.fc = nn.Linear(latent_dim, 512*4*4)
      self.deconv = nn.Sequential(
         nn.ConvTranspose2d(512, 256, 4, 2, 1), nn.BatchNorm2d(256), nn.ReLU(),
         nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.ReLU(),
         nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),
         nn.ConvTranspose2d(64, 3, 4, 2, 1), nn.Tanh()
```

```
        )

    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)
        return self.deconv(x)
```

Step 5: Define the VAE Model

```
class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar
```

Step 6: Define Loss Function

```
def vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(recon_x, x, reduction='sum')
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div
```

Step 7: Train the Model and Generate Images

```
model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for images, _ in dataloader:
        images = images.to(device)
        recon, mu, logvar = model(images)
        loss = vae_loss(recon, images, mu, logvar)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()
```

```
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {total_loss/len(dataloader.dataset):.4f}")

    # Generate and visualize samples
    model.eval()
    with torch.no_grad():
        z = torch.randn(64, latent_dim).to(device)
        sample_images = model.decoder(z).cpu() * 0.5 + 0.5  # De-normalize
        grid = torchvision.utils.make_grid(sample_images, nrow=8)
        plt.imshow(grid.permute(1, 2, 0))
        plt.axis('off')
        plt.title(f"Generated Faces - Epoch {epoch+1}")
        plt.show()
```

Conclusion:

In this experiment, a Variational Autoencoder was implemented and trained on a subset of the CelebA dataset. The model effectively learned to encode facial features into a latent space and generate realistic synthetic faces from random samples in that space. This showcases the generative power of deep neural architectures.

EX 9 TEXT GENERATION USING LSTM NETWORKS

DATE:

Problem Statement:

Build a text generation model using Long Short-Term Memory (LSTM) networks. Train the model on a text corpus to generate coherent sequences of text and evaluate the output for fluency and coherence.

Suggested Dataset: Shakespeare Corpus

Objectives:

1. Understand sequential modeling for natural language generation.
2. Train a character-level LSTM model to learn language patterns.
3. Generate text using a seed prompt and evaluate the results.
4. Analyze the fluency and creativity of LSTM-generated outputs.

Scope:

Text generation is a foundational task in natural language processing. This experiment demonstrates how LSTMs can learn syntactic and semantic patterns over time and generate believable sequences of text. The use of character-level modeling helps capture detailed language structures.

Tools and Libraries Used:

1. Python 3.x
2. TensorFlow / Keras
3. NumPy
4. Shakespeare Text Corpus (Tiny Shakespeare)

Implementation Steps:

Step 1: Load and Preprocess the Dataset

```
import tensorflow as tf
import numpy as np

text = tf.keras.utils.get_file('shakespeare.txt',
   'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt')
text = open(text, 'r').read().lower()
chars = sorted(set(text))
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}
```

Step 2: Create Input and Output Sequences

```
seq_len = 40
X = []
y = []

for i in range(len(text) - seq_len):
   input_seq = text[i:i + seq_len]
   target_char = text[i + seq_len]
   X.append([c2i[c] for c in input_seq])
   y.append(c2i[target_char])

X = np.array(X)
y = np.array(y)
```

Step 3: Build the LSTM Model

```
model = tf.keras.Sequential([
   tf.keras.layers.Embedding(len(chars), 64, input_length=seq_len),
   tf.keras.layers.LSTM(128),
   tf.keras.layers.Dense(len(chars), activation='softmax')
])
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam')
model.fit(X, y, batch_size=128, epochs=1)
```

Step 4: Define the Text Generation Function

```
def generate(seed, length=300):
   seq = [c2i[c] for c in seed.lower()]
   for _ in range(length):
      inp = np.array(seq[-seq_len:]).reshape(1, -1)
      pred = model.predict(inp, verbose=0)[0]
      next_idx = np.random.choice(len(pred), p=pred)
      seq.append(next_idx)
   return seed + ''.join(i2c[i] for i in seq[len(seed):])
```

Step 5: Generate and Display Text

```
print("\nGenerated Text:\n")
print(generate("shall i compare thee to a summer's day?\n"))
```

Conclusion:

This experiment demonstrates how an LSTM model learns to predict and generate sequences of text based on character-level inputs. Despite being trained for a single epoch, the generated output showcases structural fluency and stylistic hints of Shakespearean English. Further training can improve coherence and creativity.

EX 10 IMAGE GENERATION USING GENERATIVE ADVERSARIAL NETWORK (GAN)

DATE:

Problem Statement:

Train a Generative Adversarial Network (GAN) to generate new images from a dataset. Evaluate the quality of generated images using both visual inspection and a quantitative metric such as Inception Score (IS) or Fréchet Inception Distance (FID).

Suggested Dataset: LSUN Dataset

Objectives:

1. Understand the architecture of GANs and their adversarial training process.
2. Use StyleGAN2-ADA, a state-of-the-art GAN, for high-quality image generation.
3. Evaluate generated images using IS and FID metrics.
4. Compare generated outputs with real samples and assess realism.

Scope:

GANs are powerful generative models used in image synthesis, super-resolution, style transfer, and more. This experiment familiarizes students with modern GAN architectures and helps develop understanding in both visual and metric-based evaluation of generative quality.

Tools and Libraries Used:

1. Python 3.x
2. PyTorch
3. torchvision
4. StyleGAN2-ADA
5. torchmetrics
6. FID & Inception Score libraries
7. CelebA Dataset

Implementation Steps:

Step 1: Clone StyleGAN2-ADA Repository

```
REPO = 'stylegan2-ada-pytorch'
REPO_URL = 'https://github.com/NVlabs/stylegan2-ada-pytorch.git'

def clone_repo():
  if not os.path.exists(REPO):
    subprocess.run(['git', 'clone', REPO_URL])
```

```
sys.path.append(os.path.abspath(REPO))
```

Step 2: Download Pretrained Generator

```
MODEL_URL = 'https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-pytorch/pretrained/ffhq.pkl'
MODEL_PATH = 'ffhq.pkl'

def download_model():
    if not os.path.exists(MODEL_PATH):
        import urllib.request
        urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)
```

Step 3: Generate Synthetic Images

```
def generate_images():
    import legacy
    import dnnlib
    G = legacy.load_network_pkl(open(MODEL_PATH, 'rb'))['G_ema'].to(device)
    os.makedirs('generated_images', exist_ok=True)
    imgs = []
    for i in range(1):  # NUM_IMAGES = 1
        z = torch.randn([1, G.z_dim], device=device)
        label = torch.zeros([1, G.c_dim], device=device)
        img = (G(z, label, truncation_psi=0.5, noise_mode='const') + 1) * 0.5
        save_image(img, f'generated_images/fake_{i}.png')
        imgs.append(img.cpu())
    return torch.cat(imgs)
```

Step 4: Calculate Inception Score

```
from torchmetrics.image.inception import InceptionScore

def calculate_inception_score(imgs):
    loader = DataLoader(TensorDataset((imgs * 255).clamp(0, 255).to(torch.uint8)), batch_size=32)
    is_metric = InceptionScore()
    for batch in loader:
        is_metric.update(batch[0])
    mean, std = is_metric.compute()
    print(f"Inception Score: {mean:.2f} ± {std:.2f}")
```

Step 5: Prepare Real Images from CelebA

```
def prepare_real_images():
    dataset = dsets.CelebA(root='./data', split='train',
        transform=transforms.Compose([
            transforms.Resize(512),
            transforms.CenterCrop(512),
            transforms.ToTensor()
        ]),
```

```
        download=True)
    loader = DataLoader(dataset, batch_size=1, shuffle=True)
    os.makedirs('real_images', exist_ok=True)
    for i, (img, _) in enumerate(loader):
        if i >= 1: break
        save_image(img, f"real_images/real_{i}.png")
```

Step 6: Compute FID Between Real and Generated Images

```
def calculate_fid():
    subprocess.run(["python", "-m", "pytorch_fid", "real_images", "generated_images"])
```

Step 7: Run Entire Workflow

```
def main():
    clone_repo()
    download_model()
    imgs = generate_images()
    calculate_inception_score(imgs)
    prepare_real_images()
    calculate_fid()

if __name__ == "__main__":
    main()
```

Conclusion:

This experiment highlights the capabilities of advanced GANs like StyleGAN2 in generating photorealistic images. The use of Inception Score and FID provides quantitative support to visual assessments. GANs remain a cornerstone in generative deep learning research, with wide applications in media, design, and beyond.

VIVA QUESTIONS

1. What is a neural network?

A neural network is a machine learning model inspired by the human brain, consisting of layers of interconnected nodes that process input data.

2. What is backpropagation?

Backpropagation is an algorithm used to train neural networks by propagating the error from the output back through the layers to update weights.

3. What is the purpose of the sigmoid activation function?

The sigmoid function introduces non-linearity and maps input values between 0 and 1.

4. Why do we normalize input images?

Normalization ensures that inputs fall within a consistent range, speeding up training and improving convergence.

5. What is one-hot encoding?

One-hot encoding converts categorical labels into binary vectors to be used in classification tasks.

6. How is cross-entropy loss calculated?

Cross-entropy loss measures the dissimilarity between predicted probabilities and actual labels, commonly used in classification.

7. What happens if the learning rate is too high?

A high learning rate may cause the model to overshoot minima and fail to converge.

8. What is the role of hidden layers in a neural network?

Hidden layers extract complex features and learn patterns in the input data.

9. What is the vanishing gradient problem?

It occurs when gradients become too small to update weights effectively, especially with deep networks and sigmoid activation.

10. Why is MNIST used for digit recognition?

MNIST is a well-known, simple dataset of handwritten digits ideal for benchmarking image classification algorithms.

11. What is a perceptron?

A perceptron is a basic unit in a neural network that performs a weighted sum followed by an activation function.

12. How is an MLP different from a simple neural network?

An MLP contains multiple hidden layers, allowing it to learn more complex patterns.

13. What is the advantage of using ReLU over sigmoid?

ReLU is computationally efficient and avoids vanishing gradient issues seen with sigmoid.

14. What is validation accuracy?

Validation accuracy measures the model's performance on unseen data during training to check for generalization.

15. Why do we use multiple hidden layers?

To model hierarchical features and complex relationships in the data.

16. What is batch size?

Batch size is the number of training samples processed before the model updates weights.

17. What is overfitting?

Overfitting is when a model learns noise from the training data and performs poorly on unseen data.

18. How do we prevent overfitting?

Using dropout, regularization, or increasing training data helps prevent overfitting.

19. What is the purpose of an activation function?

It adds non-linearity, allowing the network to solve complex tasks.

20. Why is softmax used in the output layer?

Softmax converts raw scores into probability distributions for multi-class classification.

21. What is Stochastic Gradient Descent?

SGD updates weights using one or a few training examples at a time, allowing fast updates.

22. What is momentum in SGD?

Momentum helps accelerate SGD in the right direction and dampens oscillations.

23. How does Adam differ from SGD?

Adam uses adaptive learning rates and momentum, typically converging faster.

24. Why do we compare optimizers?

To find which optimizer performs best on a specific task or dataset.

25. What is convergence in deep learning?

Convergence refers to the point where further training results in minimal improvement in loss.

26. What is learning rate decay?

A strategy to reduce learning rate during training to improve fine-tuning and stability.

27. What are the benefits of Adam?

Adam is adaptive, efficient, and works well across many datasets and problems.

28. What does the loss function indicate?

It measures the difference between predicted and actual values during training.

29. Why is CIFAR-10 used for classification?

CIFAR-10 is a popular benchmark dataset for small image classification tasks.

30. What does a lower loss value indicate?

It indicates better performance and improved prediction accuracy.

31. What is a Convolutional Neural Network?

A CNN is a neural network architecture specialized for image data, using convolution layers.

32. What is a convolutional layer?

It applies filters to extract local features like edges or textures from input images.

33. What does max pooling do?

Max pooling reduces dimensionality by selecting the highest value from a patch of the feature map.

34. What is the role of ReLU in CNNs?

ReLU introduces non-linearity, allowing CNNs to model complex data.

35. What is a kernel in a CNN?

A kernel is a small matrix that slides over the input to perform convolution.

36. Why use stride and padding?

Stride controls the movement of the kernel; padding maintains spatial dimensions.

37. What is the purpose of flattening?

Flattening converts the 2D feature maps into 1D input for fully connected layers.

38. What is feature extraction?

It refers to identifying informative features from images for classification.

39. Why are CNNs preferred for image data?

CNNs capture spatial hierarchies and reduce parameters compared to dense networks.

40. What is the receptive field?

The receptive field is the region of the input image affecting a specific output neuron.

41. What is transfer learning?

It involves using pretrained models on new tasks to reduce training time and improve performance.

42. How does VGG differ from ResNet?

VGG uses deep sequential layers, while ResNet includes skip connections to mitigate degradation.

43. What is a residual connection?

It allows input to bypass certain layers, helping gradients flow more easily in deep networks.

44. Why is GoogLeNet called an Inception network?

It uses inception modules that combine multiple convolutions in one block.

45. What is fine-tuning?

Fine-tuning updates pretrained weights on a new task to improve performance.

46. Why use pretrained models?

They reduce computational cost and require less data for training.

47. What is ImageNet?

A large image dataset used to train and benchmark deep learning models.

48. What is the benefit of deeper networks?

Deeper networks can model more abstract and complex patterns.

49. What is model generalization?

The ability of a model to perform well on unseen data.

50. What does accuracy indicate in classification tasks?

Accuracy measures the percentage of correctly classified samples.

51. What is a Recurrent Neural Network?
An RNN processes sequential data by maintaining hidden states over time.

52. What is a BiRNN?
A Bidirectional RNN processes input sequences in both forward and backward directions.

53. What type of data are RNNs suited for?
RNNs are ideal for sequential or time-series data.

54. Why does FFNN fail on sequential data?
FFNNs do not preserve temporal dependencies between inputs.

55. What is hidden state in RNN?
It stores information from previous time steps to influence future outputs.

56. What is sequence prediction?
It involves predicting future elements in a time-series using historical data.

57. What is the main advantage of BiRNN?
BiRNNs leverage context from both past and future in a sequence.

58. What is sequence length?
The number of time steps considered in each input sample.

59. Why is MSE used in time-series prediction?
MSE penalizes large errors and is standard for regression-based tasks.

60. What is the role of re-scaling or normalization?
It ensures that input features contribute equally during training.

61. What is image captioning?
It is the task of generating descriptive text for a given image.

62. What is the role of CNN in image captioning?
The CNN extracts features from the image, which are used to inform caption generation.

63. What is the role of RNN in captioning?
The RNN takes image features and sequentially generates a sentence.

64. Why is MS COCO used for image captioning?
MS COCO contains images with multiple human-annotated captions, ideal for training.

65. What is the encoder-decoder architecture?
A structure where the encoder processes input (image), and decoder generates output (text).

66. What is beam search?
An advanced decoding strategy to generate more coherent and optimal sequences.

67. What is a transformer model?
A transformer is an attention-based model replacing RNNs in many sequence tasks.

68. Why use pre-trained models like ViT-GPT2?
They combine strong vision and language capabilities for efficient captioning.

69. What is a token in NLP?
A token is a unit (character, word, or subword) processed by a language model.

70. What is the significance of max_length in generation?
It limits the number of tokens in the generated sequence to control output size.

71. What is a VAE?
A VAE is a generative model that learns to encode data into a distribution and sample new data from it.

72. How is a VAE different from a traditional autoencoder?
VAEs generate new data by sampling from a latent distribution, not just reconstructing input.

73. What is latent space?
A compressed representation of input data from which new samples can be generated.

74. What is KL divergence?
KL divergence measures how one probability distribution diverges from another.

75. What is reparameterization?
It allows gradients to flow through stochastic nodes by using a deterministic function.

76. What does the decoder do in a VAE?
The decoder reconstructs the original image from a latent vector.

77. Why use batch normalization?
To stabilize and speed up training by normalizing layer outputs.

78. What is the role of Tanh in the output layer?
Tanh ensures output pixels are in the range [-1, 1], useful for image reconstruction.

79. How does a VAE generate new images?
By sampling latent vectors from a normal distribution and decoding them.

80. What is the benefit of using CelebA?
It is a large-scale face dataset ideal for generative image tasks.

81. What is LSTM?
LSTM is a type of RNN that overcomes vanishing gradients using memory cells.

82. What is a character-level language model?
It predicts the next character in a sequence rather than full words.

83. What is the input to an LSTM in this task?
A sequence of characters encoded as integers.

84. What does the embedding layer do?
It maps characters to dense vector representations before feeding them into LSTM.

85. What is the purpose of sampling in generation?
Sampling allows choosing the next character based on probabilities instead of always picking the highest.

86. What is the temperature in text generation?
It controls randomness in sampling; higher values yield more creative outputs.

87. What is sparse categorical cross-entropy?
A loss function used when labels are integers instead of one-hot vectors.

88. Why use Shakespeare for training?
Shakespeare's corpus offers structured and stylized text for evaluating generation quality.

89. What is a seed sequence?
The initial input text used to prime the model for text generation.

90. What is the impact of model depth on text generation?
Deeper models can capture more complex patterns but may overfit.

91. What is a GAN?
A GAN consists of a generator and discriminator trained adversarially to generate realistic data.

92. What is the generator's role?
The generator creates fake images from random noise.

93. What is the discriminator's role?
The discriminator distinguishes real images from fake ones.

94. What is adversarial training?
A process where two models compete, improving each other iteratively.

95. What is the Inception Score (IS)?
A metric to evaluate generated image quality using confidence and diversity.

96. What is Fréchet Inception Distance (FID)?
FID compares statistics of real and generated images using Inception features.

97. What is StyleGAN2?
A state-of-the-art GAN architecture known for generating high-quality images.

98. Why is truncation used in GANs?
Truncation improves sample quality by controlling variability in latent vectors.

99. What are mode collapse issues?
When the generator produces limited variety in outputs, reducing diversity.

100. Why is CelebA used as real data in this experiment?
CelebA provides real facial images to compare against generated outputs for FID computation.

REFERENCES

- Neural Networks and Backpropagation: https://cs231n.github.io/neural-networks-1/
- ReLU Activation Function: https://en.wikipedia.org/wiki/Rectifier_(neural_networks)
- One-Hot Encoding Concept: https://machinelearningmastery.com/why-one-hot-encode-data-in-machine-learning/
- Loss Functions in Deep Learning: https://keras.io/api/losses/
- Multi-Layer Perceptrons (MLP): https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- Adam and SGD Optimizers: https://towardsdatascience.com/adam-latest-trends-in-deep-learning-optimization-6be9a291375c
- Convolutional Neural Networks (CNNs): https://cs231n.github.io/convolutional-networks/
- PyTorch Documentation: https://pytorch.org/docs/stable/index.html
- VGG, ResNet, GoogLeNet Architectures: https://paperswithcode.com/methods/category/convolutional-neural-networks
- Recurrent Neural Networks (RNN): https://colah.github.io/posts/2015-08-Understanding-LSTMs/
- Bidirectional RNNs: https://towardsdatascience.com/understanding-bidirectional-rnn-in-pytorch-5bd60d6fd0f6
- Inception Score and FID: https://github.com/mseitzer/pytorch-fid
- LSTM for Text Generation: https://keras.io/examples/generative/lstm_character_level_text_generation/